UNIVERSIDADE FRANCISCANA — UFN

Área de Ciências Tecnológicas | Curso de Ciência da Computação

PESQUISA AVANÇADA EM TOLERÂNCIA A FALHAS, DEPENDABILIDADE E DETECTORES DE DEFEITOS EM SISTEMAS DISTRIBUÍDOS

Uma Investigação Formal sobre o Teorema FLP, Padrões de Qualidade ISO/IEC 25010 e as Classes de Detectores Não Confiáveis de Chandra & Toueg

Disciplina:	Tolerância a Falhas
Professor:	Rafael Bisogno
Trabalho:	Trabalho Acadêmico T2 — Pesquisa Conceitual e Prática
Instituição:	Universidade Franciscana (UFN), Santa Maria - RS

Santa Maria — RS

2026

1. Detectores de Defeitos (DF)

1.1 Contextualização em Sistemas Distribuídos: Em arquiteturas computacionais centralizadas, o monitoramento do ciclo de vida dos processos é garantido deterministicamente pelo núcleo do sistema operacional por interrupção de hardware ou sinais POSIX. Em contrapartida, sistemas distribuídos operam sob a ausência de memória compartilhada e de um relógio físico global perfeito. Os nós comunicam-se exclusivamente via troca assíncrona de mensagens sobre canais sujeitos a latências imprevisíveis, perda de pacotes e partições de rede. Nesse cenário hostil, a constatação do estado operacional de nós remotos é um dos desafios basilares da computação distribuída, motivando o surgimento dos *Detectores de Defeitos (Failure Detectors - DF)*, formalizados pioneiramente por Chandra e Toueg (1996).

1.2 Definição de Detector de Defeitos: Operacionalmente, um Detector de Defeitos é uma abstração oracular de software distribuído acoplada localmente a cada nó participante. O módulo local interroga periodicamente ou escuta os demais nós do cluster, produzindo como saída contínua uma *Lista de Suspeitos* contendo os processos com suspeita de falha. Trata-se de um mecanismo passivo de consulta, cuja saída é expressa matematicamente como um subconjunto de processos sob suspeita no instante atual: $D_p(t) \subseteq \Pi$.

1.3 Processos Falhos: Na teoria clássica de tolerância a falhas, um processo é denominado *correto* se executa suas transições de estado estritamente de acordo com o algoritmo ao longo de toda a computação. Inversamente, é rotulado como *falho* caso desvie dessa especificação. No escopo primordial dos detectores de defeitos, adota-se o modelo de falha por colapso (*crash-stop*), em que o processo funciona perfeitamente até determinado instante e, a partir dele, cessa de forma definitiva a execução de instruções e o envio de qualquer mensagem.

1.4 Distinção Fundamental: Processo Lento versus Processo Falho: A ambiguidade central de sistemas assíncronos reside na impossibilidade de diferenciar, pelo silêncio de canal, se um nó remoto entrou em colapso irreversível (*crashed*) ou se está meramente hiper-lento (*slow*). Condições reais de infraestrutura — tais como longas pausas de Garbage Collection em Java/Go, sobrecarga severa de CPU, swapping de memória em disco ou filas de roteadores saturadas — provocam ausência temporária de resposta que mimetiza com exatidão a morte física de um nó.

1.5 Relação com Consenso: O problema do Consenso exige que nós proponham valores e decidam conjuntamente sobre um único resultado consistente. Protocolos de consenso (como Raft, Paxos e Zab) dependem diretamente da detecção de falhas para contornar nós paralisados e formar quóruns de maioria com os sobreviventes. Sem detectores, o consenso arrisca o travamento perpétuo.

1.6 Propriedades de Completude e Exatidão: Chandra e Toueg (1996) demonstraram que a eficácia de qualquer detector é decomposta em duas propriedades ortogonais: *Completude (Completeness)*, que governa a capacidade de identificar processos verdadeiramente falhos (assegurando progresso / liveness), e *Exatidão (Accuracy)*, que delimita a ocorrência de suspeitas falsas sobre processos sadios (resguardando a segurança / safety).

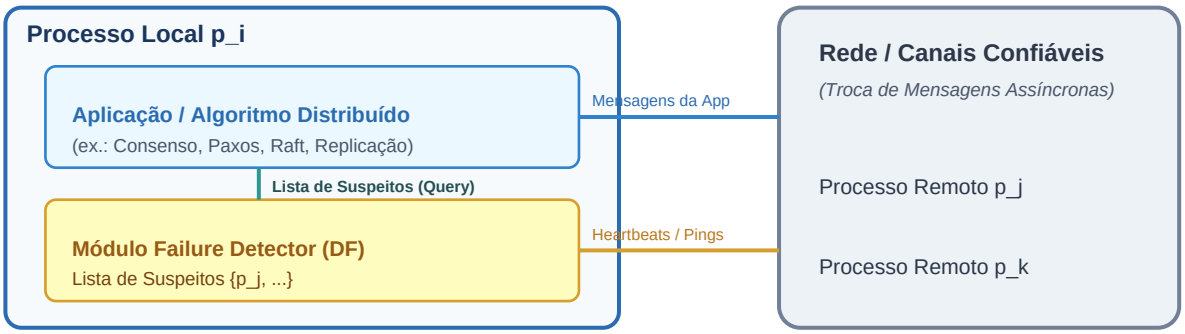

Figura 1: Arquitetura do Módulo Local de Detecção de Falhas em um Processo Distribuído.

Fonte: Adaptado diretamente de CHANDRA, Tushar D.; TOUEG, Sam. *Unreliable Failure Detectors for Reliable Distributed Systems*, JACM, v. 43, n. 2, p. 228, Fig. 1, 1996. doi:10.1145/226643.226647

2. Qualidade de Software e Dependabilidade

2.1 Conceituação de Qualidade de Software: Qualidade de software é o grau em que um sistema atende aos requisitos explícitos e expectativas tácitas de confiabilidade, desempenho e estabilidade de suas partes interessadas. Em ambientes distribuídos, o conceito transcende a ausência de bugs no código de aplicação e abrange a resiliência arquitetural e continuidade de negócio.

2.2 Da ISO/IEC 9126 para a ISO/IEC 25010 (SQuaRE): A histórica norma ISO/IEC 9126 (1991) estabelecia seis atributos (Funcionalidade, Confiabilidade, Usabilidade, Eficiência, Manutenibilidade e Portabilidade). Em 2011, foi revogada e sucedida pela ISO/IEC 25010 (série SQuaRE), expandindo o modelo para 8 características primárias, dividindo a Confiabilidade em quatro subcaracterísticas essenciais: Maturidade, Disponibilidade, Tolerância a Falhas e Recuperabilidade.

2.3 Relação dos Atributos Sistêmicos com a Dependabilidade: A dependabilidade (*dependability*), formalizada por Jean-Claude Laprie e refinada em Avižienis et al. (2004), representa a capacidade de confiar justificadamente no serviço fornecido por um sistema computacional. Ela consolida atributos como Confiabilidade, Disponibilidade, Segurança Operacional (Safety), Integridade e Manutenibilidade, estruturando as ameaças (defeitos -> erros -> falhas) e os meios para superá-las (tolerância, prevenção, remoção e previsão).

2.4 A Dependabilidade Viabilizada por Detectores de Defeitos: Os Detectores de Defeitos operam como os 'olhos' do sistema de dependabilidade. Sem eles, quando um servidor colapsa em uma esteira de microsserviços ou cluster em nuvem, os orquestradores não conseguem desviar o fluxo de tráfego nem rebalancear réplicas, levando o cluster a falhas em cascata e degradação severa da disponibilidade.

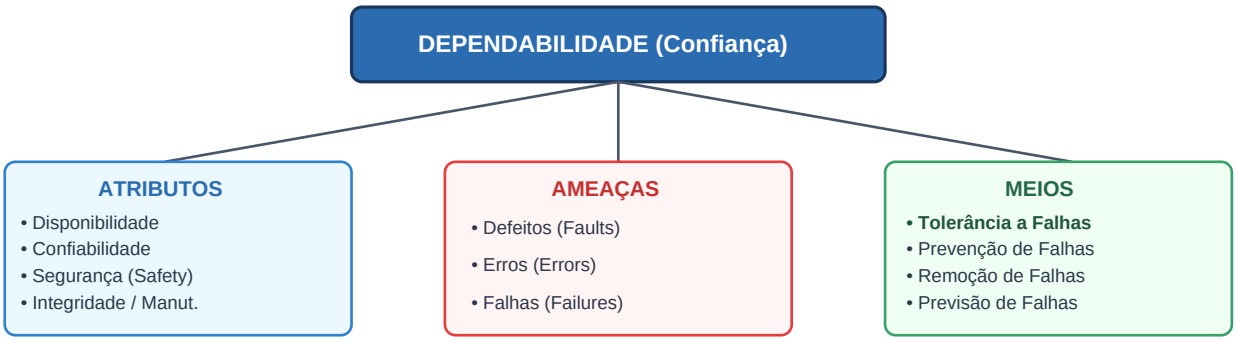

Figura 2: A Árvore da Dependabilidade: Atributos, Ameaças e Meios.

Fonte: Reproduzido e adaptado de AVIŽIENIS, Algirdas; LAPRIE, Jean-Claude et al. *Basic Concepts and Taxonomy of Dependable and Secure Computing*, IEEE TDSC, v. 1, n. 1, p. 13, Fig. 1, 2004. doi:10.1109/TDSC.2004.2

3. Modelo de Características de Qualidade Aplicado a Sistemas Distribuídos

3.1 Análise das Subcaracterísticas da ISO/IEC 25010: A norma ISO 25010 delineia quatro pilares de confiabilidade diretamente relevantes para arquiteturas tolerantes a falhas:

Subcaracterística	Definição Formal ISO 25010	Manifestação em Sistemas Distribuídos
Maturidade	Grau em que o sistema atende às necessidades de confiabilidade sob operação normal.	Baixa taxa de crash, ausência de memory leaks assíncronos e estabilidade frente a picos de tráfego.
Disponibilidade	Proporção de tempo em que o serviço está acessível e operacional para uso.	Métricas de SLA/SLO (ex.: 99.999% ou 'cinco noves'), failover rápido de balanceadores e tolerância a partições.
Tolerância a Falhas	Capacidade de operar como planejado apesar da ocorrência de falhas de hardware/software.	Capacidade de tolerar a queda de f nós em clusters de 2f + 1 nós via consenso e quóruns de voto.

Recuperabilidade	Capacidade de restabelecer o serviço e recuperar dados íntegros após uma falha.	Replay de Write-Ahead Log (WAL), reconstrução de estado a partir de snapshots e MTTR reduzido.

Fonte: Elaborado com base nas especificações oficiais da norma ISO/IEC 25010:2011 (disponível em iso.org/standard/35733.html).

4. Técnicas Fundamentais de Tolerância a Falhas

A engenharia de sistemas distribuídos sustenta sua resiliência por meio de técnicas consolidadas na literatura clássica (Coulouris et al., 2013; Tanenbaum & Van Steen, 2007):

4.1 Redundância: O uso de recursos adicionais é estruturado em quatro eixos: *Espacial* (duplicação de servidores, rotas de rede e zonas de disponibilidade); *Temporal* (retransmissões idempotentes com exponential backoff); *Informativa* (bits adicionais como códigos Reed-Solomon, Erasure Coding e checksums); e *Modular (TMR)* (votação por maioria em saídas triplicadas).

4.2 Atualização e Restauração de Estados Consistentes: Para evitar o desastroso 'efeito dominó' em falhas, sistemas empregam *Checkpointing Coordenado* (gerando snapshots globais consistentes via algoritmo de Chandy-Lamport) e *Write-Ahead Logging (WAL)*, onde qualquer transação é persistida de modo síncrono em log sequencial durável antes de ser refletida na memória volátil.

4.3 Comunicação em Grupo Confiável: Estabelece primitivas como *Reliable Multicast* (garantindo que se um nó correto entrega a mensagem, todos os corretos a entregam) e o modelo de *Sincronia Virtual (Virtual Synchrony)* introduzido por Ken Birman no sistema ISIS, sincronizando mudanças de membros no grupo com as fronteiras de entrega de mensagens.

4.4 Replicação de Recursos: Divide-se em *Replicação Passiva (Primary-Backup)*, onde um líder executa escritas e propaga logs aos seguidores, e *Replicação Ativa (State Machine Replication - SMR)*, onde todas as réplicas processam de modo determinístico as mesmas entradas na mesma ordem.

4.5 Ordenação de Eventos: Como relógios físicos sofrem desvios de sincronia (clock drift), a consistência é mantida por meio de *Relógios Lógicos de Lamport* (ordenação causal 'happened-before') e *Difusão Atômica (Total Order Broadcast)*, primitivas que asseguram que todas as réplicas processem exatamente a mesma sequência de comandos.

5. Impossibilidade FLP — Conceito e Fundamentação

5.1 O Teorema de Fischer, Lynch e Paterson (1985): Em abril de 1985, Fischer, Lynch e Paterson publicaram o célebre artigo *'Impossibility of Distributed Consensus with One Unreliable Process'* (Journal of the ACM, v. 32, n. 2), consagrado com o Prêmio Dijkstra. O teorema enuncia formalmente:

> *'Em um sistema distribuído puramente assíncrono, nenhum algoritmo determinístico é capaz de garantir Consenso na presença de mesmo uma única falha de processo por colapso (crash-stop).'*

5.2 O Sistema Distribuído Assíncrono: O modelo FLP pressupõe ausência de limites superiores conhecidos para atrasos de rede e tempos de processamento. Mensagens são confiáveis (nunca se corrompem e eventualmente são entregues), mas podem levar intervalos de tempo arbitrariamente longos para trafegar.

5.3 Falha por Colapso (Crash): A falha permitida é a mais simples e não-maliciosa: o nó executa normalmente e para permanentemente. Demonstrar a impossibilidade sob o tipo mais suave de falha prova a extrema severidade do problema.

6. Impossibilidade FLP — Causa Raiz e Consequências Práticas

6.1 A Indistinção do Silêncio e a Bivalência: A base matemática da prova reside na existência de configurações *bivalentes* (onde o resultado final pode ser tanto 0 quanto 1). Em uma rede assíncrona, se um nó crucial silencia, o protocolo enfrenta um dilema insolúvel: se prosseguir sem o nó, arrisca violar a Integridade (Safety) caso o nó estivesse vivo e decidisse de forma concorrente; se esperar pela resposta, arrisca violar o Progresso (Liveness) caso o nó tenha morrido. Um agendador de rede adverso pode manipular a ordem de entrega das mensagens de modo a manter o sistema perpetuamente bivalente.

6.2 Espera Indefinida, Bloqueio e Impacto Sistêmico: Protocolos ingênuos (como o Two-Phase Commit / 2PC) sofrem bloqueio sistemático (deadlock de recursos). Para superar o FLP no mundo real, sistemas adotam três alternativas: (a) sincronia parcial (Dwork et al., 1988); (b) algoritmos probabilísticos/randomizados; ou (c) **Detectores de Defeitos Não Confiáveis** (Chandra & Toueg, 1996).

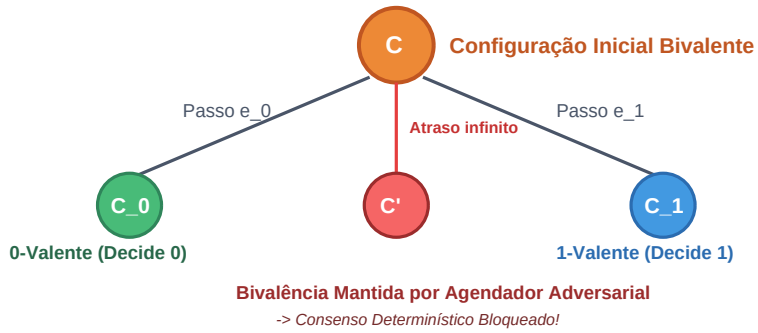

Figura 3: Árvore de Configurações e o Dilema da Bivalência no Teorema FLP.

Fonte: Reproduzido e adaptado de FISCHER, Michael J.; LYNCH, Nancy A.; PATERSON, Michael S. *Impossibility of Distributed Consensus with One Unreliable Process*, JACM, v. 32, n. 2, p. 376, 1985. doi:10.1145/3149.214121

7. Superando o FLP: A Abordagem dos Detectores de Defeitos

7.1 Desacoplamento do Tempo Físico: A inovação revolucionária de Chandra e Toueg consistiu em retirar temporizadores do algoritmo de consenso e isolá-los dentro de um componente externo — o Detector de Defeitos. O consenso em si permanece matematicamente assíncrono. O detector pode cometer erros transitórios, mas fornece ao consenso pistas suficientes para viabilizar o término com segurança.

7.2 Módulo Local e a Lista de Suspeitos: Cada processo consulta seu módulo local de DF para saber quem está na *Lista de Suspeitos*. Se um nó sob suspeita está atrasando uma decisão de quórum, os nós restantes podem ignorá-lo e prosseguir com a rodada.

7.3 Detectores Não Confiáveis (Unreliable): Em redes reais, detectores perfeitos são impossíveis. Portanto, o modelo assume detectores não confiáveis: se um nó sadio for colocado falsamente na lista de suspeitos, o algoritmo

não corrompe dados (preserva Safety); apenas uma nova rodada de votação é iniciada (impactando temporariamente Liveness).

8. Lista de Suspeitos, Percepção Momentânea e Disseminação

8.1 Remoção de Suspeitas: A lista de suspeitos é fluida. Se o nó p suspeita de q por expiração de timeout e posteriormente recebe uma mensagem tardia de q, a suspeita é cancelada e q é reabilitado no cluster.

8.2 Divergência entre Listas e Estratégias de Disseminação: Como cada nó observa a rede sob sua própria perspectiva, as listas de suspeitos de nós diferentes podem divergir. Para sincronizar visões e balancear o overhead de rede, utilizam-se técnicas avançadas:

• **All-to-All Heartbeat:** Monitoramento exaustivo onde todos pingam todos. Custo $O(N^2)$ por período; inviável em grande escala.
• **SWIM Protocol (Ping Indireto):** Quando um ping direto falha, o nó solicita a k nós auxiliares que façam pings indiretos antes de declarar suspeita (reduz falsos positivos causados por rotas locais). Custo O(1) com disseminação epidêmica O(N log N).
• **Gossip / Infecção Epidêmica:** Suspeitas e nós reabilitados são embutidos em mensagens periódicas de broadcast aleatório entre pares.

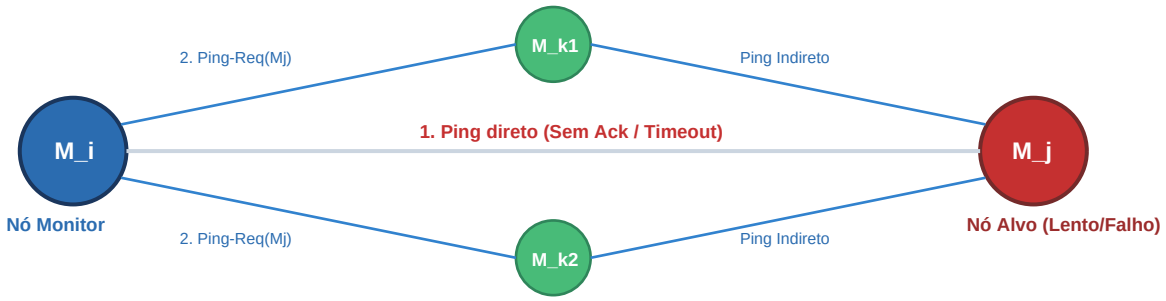

Figura 4: Protocolo SWIM: Mecanismo de Ping Direto versus Ping-Ack Indireto com k Nós Auxiliares.

Fonte: Adaptado diretamente de DAS, Abhinandan; GUPTA, Indranil; MOTIVALA, Ashish. *SWIM: A Scalable Weakly-Consistent Infection-Style Process Group Membership Protocol*, IEEE DSN, p. 305, Fig. 2, 2002. doi:10.1109/DSN.2002.1028914

9. Definição Formal do Modelo de Chandra-Toueg

O formalismo matemático estabelece um conjunto de processos $\Pi = \{p_1, ..., p_n\}$ ($|\Pi| \geq 2$) e um domínio discreto de tempo T. Canais de comunicação são confiáveis (sem duplicações infinitas, perda permanente ou mensagens corrompidas). O histórico de falhas é governado por uma função cumulativa $F: T \rightarrow 2^{\Pi}$, definindo os processos falhos (crashed(F)) e os processos corretos ($correct(F) = \Pi \setminus crashed(F)$). A saída do detector do processo p no instante t é denotada $D_p(t) \subseteq \Pi$.

10. Completude (Completeness)

A Completude rege a obrigatoriedade de identificar processos que entraram em colapso real:

• **Forte Completude (Strong Completeness):** Eventualmente, todo processo falho é permanentemente suspeito por *todos* os processos corretos: $\forall p \in crashed(F), \exists t \in T, \forall q \in correct(F), \forall t' \geq t: p \in D_q(t')$.
• **Fraca Completude (Weak Completeness):** Eventualmente, todo processo falho é permanentemente suspeito por *pelo menos um* processo correto: $\forall p \in crashed(F), \exists t \in T, \exists q \in correct(F), \forall t' \geq t: p \in D_q(t')$.

10.1 Teorema da Redução de Completude: Chandra e Toueg demonstraram que qualquer detector com Fraca Completude pode ser transformado algoritmicamente em Forte Completude sem enfraquecer seu nível de Exatidão. O mecanismo consiste em difusão confiável: sempre que um nó correto suspeita de um processo, ele transmite uma mensagem 'Eu suspeito de p'. Ao receber o aviso, os outros nós corretos adicionam p às suas respectivas listas locais de suspeitos.

11. Exatidão (Accuracy) — Níveis Iniciais e Perfeitos

A Exatidão rege o controle de erros e falsos positivos cometidos pelo detector contra processos sadios:

• **Forte Exatidão (Strong Accuracy):** Nenhum processo correto é *jamais* colocado na lista de suspeitos por nenhum processo correto: $\forall p, q \in correct(F), \forall t \in T: p \notin D_q(t)$. (Inviável em redes assíncronas reais sob latência variável).
• **Fraca Exatidão (Weak Accuracy):** Existe *pelo menos um* processo correto p* que *nunca* é colocado na lista de suspeitos por nenhum processo correto: $\exists p^* \in correct(F), \forall q \in correct(F), \forall t \in T: p^* \notin D_q(t)$. Este processo atua como uma âncora estável de liderança para coordenar propostas.

12. Exatidão Eventual (Eventual Accuracy) e o Detector Mais Fraco

Reconhecendo a inviabilidade de garantias perpétuas no início da execução, o modelo introduz propriedades *eventuais*:

• **Eventual Strong Accuracy (■P):** Existe um instante após o qual nenhum processo correto é mais suspeito por nós corretos.
• **Eventual Weak Accuracy (■S e ■W):** Existe um instante após o qual *pelo menos um* processo correto nunca mais é suspeito.

12.1 O Significado de 'Eventual' e o GST: Fundamentado na teoria de sincronia parcial (Dwork et al., 1988), 'eventual' não exige estabilidade imediata. A rede pode passar por congestionamento e falsos positivos no início. Exige-se apenas a existência de um *Tempo Global de Estabilização (GST)*, após o qual latências e processamento voltam a respeitar limites finitos, permitindo aos timeouts adaptativos cessar falsos positivos permanentemente.

12.2 O Detector Mais Fraco para Consenso: Chandra, Hadzilacos e Toueg (1996) provaram que a classe **■W** (e equivalentemente **■S**, via redução de completude) é o *detector de falhas mais fraco* para resolver Consenso em sistemas distribuídos assíncronos com maioria simples de nós corretos ($N > 2f$). Qualquer detector menos poderoso é insuficiente para assegurar consenso determinístico.

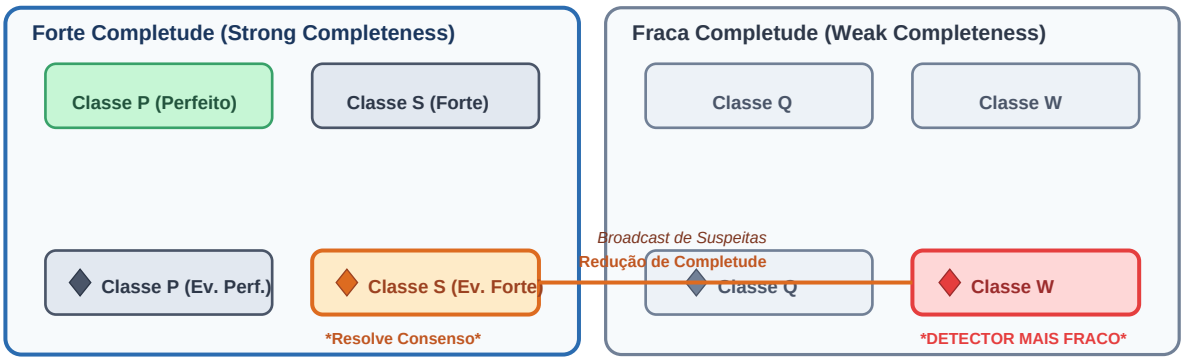

Figura 5: Hierarquia e Relações de Redutibilidade entre as 8 Classes de Chandra & Toueg (1996).

Fonte: Adaptado de CHANDRA, Tushar D.; TOUEG, Sam (1996) e CHANDRA, Tushar D.; HADZILACOS, Vassos; TOUEG, Sam. *The Weakest Failure Detector for Solving Consensus*, JACM, v. 43, n. 4, p. 685-722, 1996. doi:10.1145/234552.234562

Classe	Completude	Exatidão	Msg / Rodada	Tolerância a Falso Positivo	Aplicação na Indústria
P (Perfeito)	Forte	Forte	O(N²)	Nula (Zero erros)	Sistemas síncronos de tempo real estrito (aeroespacial).
S (Forte)	Forte	Fraca	O(N²)	Baixa (1 nó intocável)	Modelos teóricos de consenso com âncora fixa.
■P (Ev. Perf.)	Forte	Ev. Forte	O(N²)	Alta pós-GST	Failover determinístico e monitoramento em cluster local.
■S / ■W	Forte / Fraca	Ev. Fraca	O(N) a O(N log N)	**Máxima** (Ideal)	**Padrão da Indústria:** Raft, Paxos, Zookeeper, Cassandra, Consul.

Tabela 1: Matriz Comparativa Técnica das Principais Classes de Detectores de Defeitos.

12.3 Aplicações Práticas na Indústria:

• **Apache Cassandra / Akka (ϕ-Accrual Failure Detector):** Implementa a técnica de Hayashibara et al. (2004), substituindo decisões booleanas binárias por uma escala contínua de suspeita ϕ = -log10(P_later). Se ϕ ultrapassa limiares configuráveis (ex.: ϕ = 8), réplicas evitam rotear leituras para aquele nó sem deflagrar eleições destrutivas prematuras.

• **Kubernetes (NodeLease & Node Lifecycle Controller):** Cada Kubelet envia batimentos cardíacos atualizando um objeto Lease no etcd a cada 10 segundos. Se o Lease expira (timeout padrão de 40s), o nó entra em estado NotReady e os pods são reprogramados para nós íntegros.

• **HashiCorp Consul / Nomad (SWIM + Serf Gossip):** Emprega o protocolo SWIM com ping indireto e canal epidêmico de fofoca (Gossip) para monitorar nós com complexidade de mensagens constante e tolerância a partições de roteador.

13. Referências Bibliográficas

[1] AVIŽIENIS, Algirdas; LAPRIE, Jean-Claude; RANDELL, Brian; LANDWEHR, Carl. *Basic Concepts and Taxonomy of Dependable and Secure Computing*. IEEE Transactions on Dependable and Secure Computing, v. 1, n. 1, p. 11-33, 2004. DOI: 10.1109/TDSC.2004.2.

[2] CHANDRA, Tushar Deepak; TOUEG, Sam. *Unreliable Failure Detectors for Reliable Distributed Systems*. Journal of the ACM (JACM), v. 43, n. 2, p. 225-267, mar. 1996. DOI: 10.1145/226643.226647.

[3] CHANDRA, Tushar Deepak; HADZILACOS, Vassos; TOUEG, Sam. *The Weakest Failure Detector for Solving Consensus*. Journal of the ACM (JACM), v. 43, n. 4, p. 685-722, jul. 1996. DOI: 10.1145/234552.234562.

[4] COULOURIS, George; DOLLIMORE, Jean; KINDBERG, Tim; BLAIR, Gordon. *Sistemas Distribuídos: Conceitos e Projeto*. 5. ed. Porto Alegre: Bookman, 2013. ISBN: 978-8582600535.

[5] DAS, Abhinandan; GUPTA, Indranil; MOTIVALA, Ashish. *SWIM: A Scalable Weakly-Consistent Infection-Style Process Group Membership Protocol*. In: Proceedings of the International Conference on Dependable Systems and Networks (DSN'02), IEEE, Bethesda, MD, USA, 2002, p. 303-312. DOI: 10.1109/DSN.2002.1028914.

[6] DWORK, Cynthia; LYNCH, Nancy; STOCKMEYER, Larry. *Consensus in the Presence of Partial Synchrony*. Journal of the ACM (JACM), v. 35, n. 2, p. 288-323, abr. 1988. DOI: 10.1145/42282.42283.

[7] FISCHER, Michael J.; LYNCH, Nancy A.; PATERSON, Michael S. *Impossibility of Distributed Consensus with One Unreliable Process*. Journal of the ACM (JACM), v. 32, n. 2, p. 374-382, abr. 1985. DOI: 10.1145/3149.214121.

[8] HAYASHIBARA, Naohiro; DÉFAGO, Xavier; YAMIYAMA, Rami; KATAYAMA, Takuya. *The ϕ-Accrual Failure Detector: Failure Detection based on Estimation of Suspicion Level*. In: Proceedings of the 23rd IEEE International Symposium on Reliable Distributed Systems (SRDS'04), Florianópolis, Brasil, 2004, p. 66-78. DOI: 10.1109/RELDIS.2004.1353004.

[9] ISO/IEC 25010:2011. *Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — System and software quality models*. International Organization for Standardization, Genebra, Suíça, 2011. URL oficial: https://www.iso.org/standard/35733.html.

[10] ONGARO, Diego; OUSTERHOUT, John. *In Search of an Understandable Consensus Algorithm (Raft)*. In: 2014 USENIX Annual Technical Conference (USENIX ATC 14), Philadelphia, PA, 2014, p. 305-319. URL oficial: https://raft.github.io/raft.pdf.

[11] TANENBAUM, Andrew S.; VAN STEEN, Maarten. *Sistemas Distribuídos: Princípios e Paradigmas*. 2. ed. São Paulo: Pearson Prentice Hall, 2007. ISBN: 978-8576051428.

[12] APACHE CASSANDRA DOCUMENTATION. *Architecture: Failure Detection and Gossip Protocols*. Apache Software Foundation, 2024. URL: cassandra.apache.org/doc/latest/cassandra/architecture/dynamo.html.

[13] KUBERNETES OFFICIAL DOCUMENTATION. *Node Architecture: Heartbeats and Node Leases*. The Linux Foundation, 2024. URL: kubernetes.io/docs/concepts/architecture/nodes/#heartbeats.